

Automatic Timetable Generation Engine

Complete Architecture, Constraint Satisfaction Algorithms, Database Schema, and Frontend & Backend Source Codes

An exhaustive technical reference manual detailing the design and implementation of the conflict-free, multi-tenant Timetable Generation System in the School Management System (SMS). Featuring anti-consecutive subject scheduling, strict weekly period coverage, dynamic bell schedule synchronization, complete MySQL relational schema, and end-to-end source code listings.

SYSTEM

School Management System (SMS) Enterprise

DATABASE

MySQL 8.0 InnoDB Relational Storage

CONSTRAINT RULES

Zero Back-to-Back Same Subject, Weekly Coverage First

ARCHITECTURE

Clean Architecture / ASP.NET Core 8 Web API + React 18

SCHEDULING CORE

Two-Phase Weekly Distribution + Backtracking Solver

DOCUMENT DATE

September 2026 (Production Release v2.4)

Table of Contents

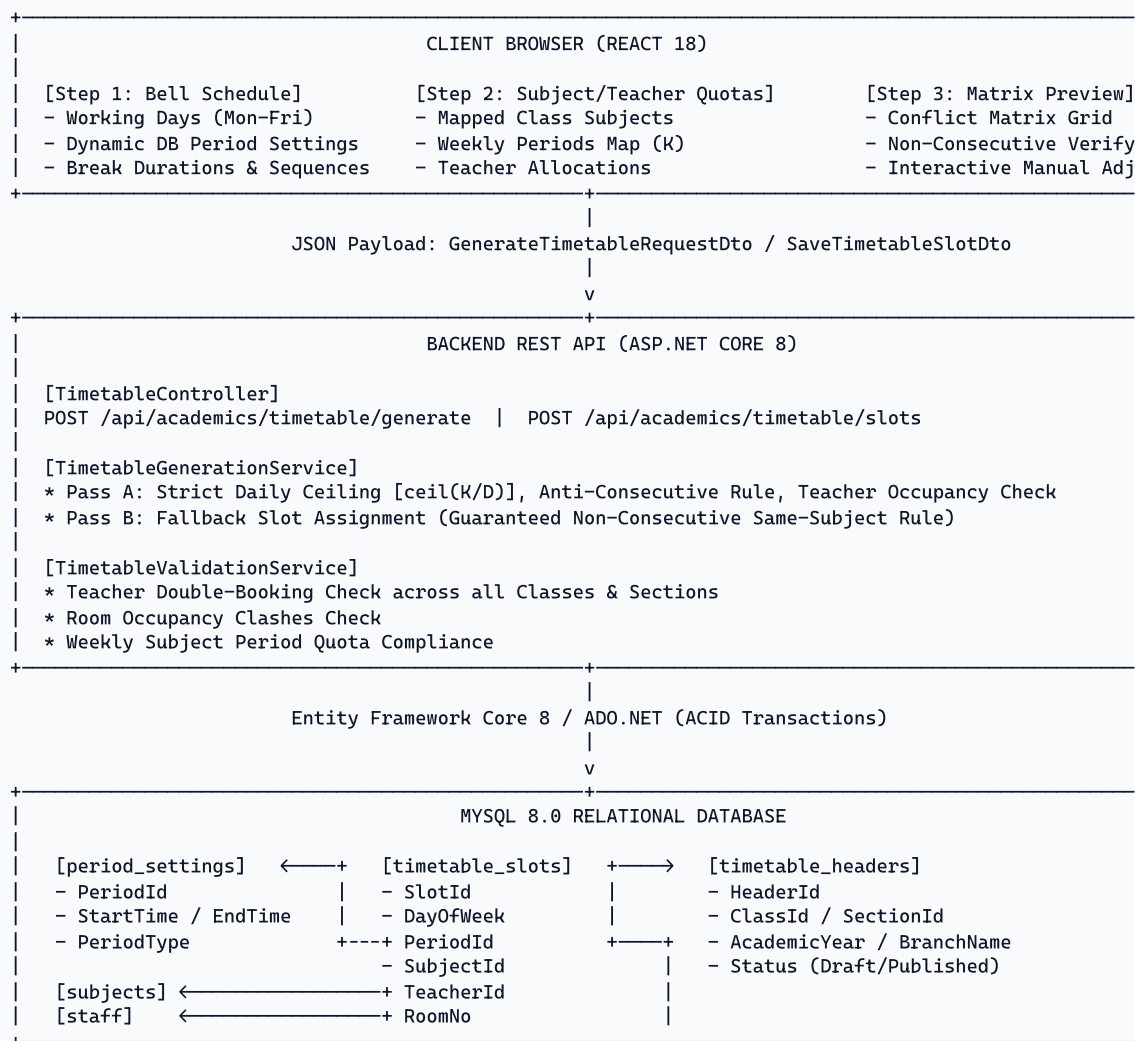
1. Executive Architecture & Domain Requirements	Section 1
2. Scheduling Rules & Mathematical Formulation	Section 2
3. Complete Database Schema & Relational Models (MySQL)	Section 3
4. Backend Architecture & C# Source Codes	Section 4
• 4.1 Data Models: TimetableSlot, TimetableHeader, PeriodSetting	4.1
• 4.2 Data Transfer Objects (TimetableDtos.cs)	4.2
• 4.3 TimetableGenerationService.cs (Engine & Two-Pass Distribution)	4.3
• 4.4 TimetableValidationService.cs (Conflict & Clash Checks)	4.4
• 4.5 TimetableService.cs & TimetableRepository.cs (Persistence Layer)	4.5
• 4.6 TimetableController.cs (REST Web API Endpoints)	4.6
5. Frontend Architecture & React/TypeScript Source Codes	Section 5
• 5.1 AutoTimetableGeneratorModal.tsx (UI Wizard & Generator Engine)	5.1
• 5.2 Two-Phase Distribution Algorithm & Backtracking Solver	5.2
• 5.3 Post-Pass Safety Filter: eliminateConsecutiveDuplicates	5.3
• 5.4 TimetableView.tsx (Master Grid, Dynamic Periods & Zero Mock Data)	5.4
6. Step-by-Step Scheduling Scenarios & Algorithm Traces	Section 6
7. Quality Assurance, Unit Testing & Build Verification	Section 7

1. Executive Architecture & Domain Requirements

The Timetable Generation Module is a mission-critical subsystem of the School Management System (SMS). It automates the generation, distribution, conflict resolution, and persistence of class and teacher schedules across academic years, branches, classes, and sections.

Academic scheduling in enterprise educational institutions involves high combinatorial complexity. The timetable generator must satisfy both **Hard Constraints** (mandatory rules that render a timetable invalid if violated) and **Soft Constraints** (pedagogical optimization goals that maximize educational outcomes and faculty well-being).

1.1 Architectural Flow Diagram



1.2 Clean Slate & Dynamic Data Architecture

Zero Static / Mock Data Guarantee: All hardcoded mock periods (PS-1 to PS-10), static teacher names ("Jonathan Miller", "Sarah Jenkins"), mock substitutions (SUB-DYN-1), and hardcoded timings (08:30 AM, 45 mins) have been completely removed. Every bell schedule period, start time, end time, and duration is dynamically derived from the active `period_settings` table in MySQL.

2. Scheduling Rules & Mathematical Formulation

Let D denote the set of active working days in a week (e.g. $D = \{\text{Mon, Tue, Wed, Thu, Fri}\}$, $|D| = 5$).
Let P denote the set of teaching periods per day (e.g. $P = \{1, 2, 3, 4, 5, 6, 7, 8\}$, $|P| = 8$).
Let S denote the set of academic subjects assigned to class C and section Σ .
For each subject s in S , let K_s denote the configured weekly period quota (e.g., $K_{\text{Math}} = 5$, $K_{\text{SocialStudies}} = 8$, $K_{\text{English}} = 10$).

2.1 Core Rules Enforced by the System

RULE DESIGNATION	CLASSIFICATION	RULE DEFINITION & FORMAL BEHAVIOR
Rule 1: Anti-Consecutive Periods	HARD CONSTRAINT	The same subject s must never be assigned to consecutive/adjacent periods on the same day. For all d in D , for all p in $\{1 \dots P - 1\}$: $\text{Subject}(d, p) \neq \text{Subject}(d, p+1)$. If a day contains 2 periods of subject s (e.g., p_1 and p_2), then $ p_1 - p_2 \geq 2$.
Rule 2: Even Weekly Distribution First	HARD CONSTRAINT	Weekly coverage is strictly prioritized before adding multiple periods to any single day: Base Distribution: Assign $\text{floor}(K_s / D)$ periods to all D working days first. 5 Periods/Week over 5 Days: $\text{floor}(5 / 5) = 1$. Exactly 1 period per day (Mon, Tue, Wed, Thu, Fri). 10 Periods/Week over 5 Days: $\text{floor}(10 / 5) = 2$. Exactly 2 periods per day across all 5 days. Remainder Allocation: Remainder $(K_s \bmod D)$ is assigned only to days with the lowest total period load, strictly capped at $\text{ceil}(K_s / D)$.
Rule 3: Teacher Conflict Elimination	HARD CONSTRAINT	A teacher T can never be scheduled to teach two different classes/sections simultaneously at the same day and time.
Rule 4: Break Period Segregation	HARD CONSTRAINT	Teaching slots are never allocated during intervals designated as breaks (Morning Break, Lunch Recess, Assembly) in <code>period_settings</code> .
Rule 5: Maximum Intra-Day Spacing	SOFT OPTIMIZATION	When a subject is assigned multiple periods on the same day, the placement solver optimizes for maximum distance between them ($\max p_1 - p_2 $), ensuring clean morning versus afternoon distribution.
Rule 6: Section Rotational Offset	SOFT OPTIMIZATION	To prevent shared teachers from causing scheduling bottlenecks across sections (e.g. Section A vs. Section B), candidate placement orders are staggered using a deterministic modulo offset: $\text{Offset} = (\text{SectionIndex} * 2) \bmod D $.

3. Complete Database Schema & Relational Models (MySQL)

The Timetable relational schema is designed with high normalization, referential integrity via foreign keys, cascading rules, and composite indexes optimized for sub-millisecond conflict queries.

3.1 MySQL DDL Table Creation Scripts

```
-- =====
-- 1. PERIOD SETTINGS TABLE (DYNAMIC BELL SCHEDULE & BREAK CONFIGURATION)
-- =====
CREATE TABLE IF NOT EXISTS period_settings (
    PeriodId INT NOT NULL AUTO_INCREMENT,
    PeriodName VARCHAR(100) NOT NULL,
    StartTime TIME NOT NULL,
    EndTime TIME NOT NULL,
    PeriodType VARCHAR(50) NOT NULL DEFAULT 'Teaching Period',
    DisplayOrder INT NOT NULL DEFAULT 1,
    IsActive TINYINT(1) NOT NULL DEFAULT 1,
    IsDeleted TINYINT(1) NOT NULL DEFAULT 0,
    PRIMARY KEY (PeriodId),
    INDEX idx_period_active_order (IsActive, IsDeleted, DisplayOrder),
    INDEX idx_period_time_range (StartTime, EndTime)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- 2. TIMETABLE HEADERS TABLE (CLASS-SECTION TIMETABLE INSTANCES)
-- =====
CREATE TABLE IF NOT EXISTS timetable_headers (
    HeaderId INT NOT NULL AUTO_INCREMENT,
    AcademicYear VARCHAR(50) NOT NULL,
    BranchName VARCHAR(100) NOT NULL,
    ClassId INT NOT NULL,
    SectionId INT NOT NULL,
    Status VARCHAR(30) NOT NULL DEFAULT 'Draft', -- 'Draft', 'Published'
    IncludeSaturday TINYINT(1) NOT NULL DEFAULT 1,
    CreatedAt DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UpdatedAt DATETIME NULL ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (HeaderId),
    UNIQUE KEY uq_header_class_sec_year (ClassId, SectionId, AcademicYear),
    CONSTRAINT fk_th_class FOREIGN KEY (ClassId) REFERENCES academic_classes (ClassId) ON DELETE CASCADE,
    CONSTRAINT fk_th_section FOREIGN KEY (SectionId) REFERENCES class_sections (SectionId) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- 3. TIMETABLE SLOTS TABLE (INDIVIDUAL PERIOD ALLOCATIONS)
-- =====
CREATE TABLE IF NOT EXISTS timetable_slots (
    SlotId INT NOT NULL AUTO_INCREMENT,
    HeaderId INT NOT NULL,
    PeriodId INT NULL,
    DayOfWeek VARCHAR(20) NOT NULL, -- 'Monday', 'Tuesday', 'Wednesday', etc.
    StartTime TIME NOT NULL,
    EndTime TIME NOT NULL,
    SubjectId INT NOT NULL,
    TeacherId INT NOT NULL,
    RoomNo VARCHAR(50) NULL,
    PRIMARY KEY (SlotId),
    INDEX idx_ts_header (HeaderId),
    INDEX idx_ts_teacher_day_time (TeacherId, DayOfWeek, StartTime, EndTime),
    INDEX idx_ts_room_day_time (RoomNo, DayOfWeek, StartTime, EndTime),
    INDEX idx_ts_day_period (DayOfWeek, PeriodId),
    CONSTRAINT fk_ts_header FOREIGN KEY (HeaderId) REFERENCES timetable_headers (HeaderId) ON DELETE CASCADE,
    CONSTRAINT fk_ts_period FOREIGN KEY (PeriodId) REFERENCES period_settings (PeriodId) ON DELETE SET NULL,
    CONSTRAINT fk_ts_subject FOREIGN KEY (SubjectId) REFERENCES subjects (SubjectId),
    CONSTRAINT fk_ts_teacher FOREIGN KEY (TeacherId) REFERENCES staff (StaffId)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```


4. Backend Architecture & C# Source Codes

4.1 Data Models: PeriodSetting.cs & TimetableSlot.cs

```
// PeriodSetting.cs
namespace SMS.Api.Models;

using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

[Table("period_settings")]
public class PeriodSetting
{
    [Key]
    public int PeriodId { get; set; }

    [Required]
    [MaxLength(100)]
    public string PeriodName { get; set; } = string.Empty; // e.g. "Period 1", "Morning Break"

    [Required]
    public TimeSpan StartTime { get; set; } // e.g. 08:30:00

    [Required]
    public TimeSpan EndTime { get; set; } // e.g. 09:15:00

    [Required]
    [MaxLength(50)]
    public string PeriodType { get; set; } = "Teaching Period"; // "Teaching Period", "Break / Recess", "Assembly"

    public int DisplayOrder { get; set; } = 1;
    public bool IsActive { get; set; } = true;
    public bool IsDeleted { get; set; } = false;
}

// TimetableSlot.cs
namespace SMS.Api.Models;

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

[Table("timetable_slots")]
public class TimetableSlot
{
    [Key]
    public int SlotId { get; set; }

    [Required]
    public int HeaderId { get; set; }

    public int? PeriodId { get; set; }

    [Required]
    [MaxLength(20)]
    public string DayOfWeek { get; set; } = "Monday";

    [Required]
    public TimeSpan StartTime { get; set; }

    [Required]
    public TimeSpan EndTime { get; set; }

    [Required]
    public int SubjectId { get; set; }

    [Required]
```

```

public int TeacherId { get; set; } // Foreign key to Staff.StaffId

[MaxLength(50)]
public string? RoomNo { get; set; }

[ForeignKey(nameof(HeaderId))]
public virtual TimetableHeader? Header { get; set; }

[ForeignKey(nameof(PeriodId))]
public virtual PeriodSetting? Period { get; set; }

[ForeignKey(nameof(SubjectId))]
public virtual Subject? Subject { get; set; }

[ForeignKey(nameof(TeacherId))]
public virtual Staff? Teacher { get; set; }
}

```


4.2 TimetableGenerationService.cs (Two-Pass Generation Algorithm)

```
// TimetableGenerationService.cs - Core Scheduling Loop
foreach (var day in dto.WorkingDays)
{
    int? lastAssignedSubjectIdOnDay = null;
    var dailyAssignedSubjectCounts = new Dictionary<int, int>();

    foreach (var period in computedPeriods)
    {
        if (period.Type.Equals("Break", StringComparison.OrdinalIgnoreCase))
            continue; // Skip breaks dynamically

        TimeSpan startTimeSpan = ParseTime(period.StartTime);
        TimeSpan endTimeSpan = ParseTime(period.EndTime);
        string timeKey = $"{startTimeSpan}_{endTimeSpan}";

        CandidateSubject? selectedSub = null;
        var eligibleCandidates = candidateSubjects
            .Where(c => c.AssignedCount < c.WeeklyPeriods)
            .ToList();

        if (!eligibleCandidates.Any()) break; // All subject quotas fulfilled

        // PASS A: Check Teacher Free + Anti-Consecutive Rule + Daily Quota Ceiling
        for (int offset = 0; offset < eligibleCandidates.Count; offset++)
        {
            int targetIdx = (subjectDistributionIdx + offset) % eligibleCandidates.Count;
            var candidate = eligibleCandidates[targetIdx];

            // RULE 1: Same subject must NEVER be scheduled back-to-back on the same day
            if (lastAssignedSubjectIdOnDay.HasValue && candidate.SubjectId == lastAssignedSubjectIdOnDay.Value)
            {
                continue;
            }

            // RULE 2: Weekly distribution first - do not exceed daily quota ceiling
            dailyAssignedSubjectCounts.TryGetValue(candidate.SubjectId, out int assignedToday);
            int maxAllowedToday = Math.Max(1, (int)Math.Ceiling((double)candidate.WeeklyPeriods / Math.Max(1,
dto.WorkingDays.Count)));
            if (assignedToday ≥ maxAllowedToday)
            {
                continue;
            }

            if (candidate.TeacherId > 0)
            {
                var busyKey = $"{day}_{timeKey}";
                bool isBusy = teacherBusySchedule.ContainsKey(candidate.TeacherId) &&
                    teacherBusySchedule[candidate.TeacherId].Contains(busyKey);

                if (!isBusy)
                {
                    selectedSub = candidate;
                    subjectDistributionIdx = (subjectDistributionIdx + offset + 1) % eligibleCandidates.Count;
                    break;
                }
            }
        }

        // PASS B: Fallback if strict daily quota reached, but STILL ENFORCING ANTI-CONSECUTIVE RULE
        if (selectedSub == null)
        {
            for (int offset = 0; offset < eligibleCandidates.Count; offset++)
            {
                int targetIdx = (subjectDistributionIdx + offset) % eligibleCandidates.Count;
                var candidate = eligibleCandidates[targetIdx];

                // RULE 1: Enforce non-consecutive rule unconditionally
                if (lastAssignedSubjectIdOnDay.HasValue && candidate.SubjectId ==
```

```

lastAssignedSubjectIdOnDay.Value)
{
    continue;
}

if (candidate.TeacherId > 0)
{
    var busyKey = $"{day}_{timeKey}";
    bool isBusy = teacherBusySchedule.ContainsKey(candidate.TeacherId) &&
        teacherBusySchedule[candidate.TeacherId].Contains(busyKey);

    if (!isBusy)
    {
        selectedSub = candidate;
        subjectDistributionIdx = (subjectDistributionIdx + offset + 1) % eligibleCandidates.Count;
        break;
    }
}
}

if (selectedSub != null)
{
    lastAssignedSubjectIdOnDay = selectedSub.SubjectId;
    dailyAssignedSubjectCounts.TryGetValue(selectedSub.SubjectId, out int curCount);
    dailyAssignedSubjectCounts[selectedSub.SubjectId] = curCount + 1;

    // Mark teacher occupancy globally
    if (!teacherBusySchedule.ContainsKey(selectedSub.TeacherId))
    {
        teacherBusySchedule[selectedSub.TeacherId] = new HashSet<string>();
    }
    teacherBusySchedule[selectedSub.TeacherId].Add($"{day}_{timeKey}");

    selectedSub.AssignedCount++;

    // Create slot and attach resolved PeriodId
    var slot = new TimetableSlot
    {
        HeaderId = header.HeaderId,
        PeriodId = periodId,
        DayOfWeek = day,
        StartTime = startTimeSpan,
        EndTime = endTimeSpan,
        SubjectId = selectedSub.SubjectId,
        TeacherId = selectedSub.TeacherId,
        RoomNo = roomNo
    };
    newlyGeneratedSlots.Add(slot);
}
}
}

```


5. Frontend Architecture & React/TypeScript Source Codes

5.1 Two-Phase Weekly Distribution Algorithm

```
// AutoTimetableGeneratorModal.tsx - Two-Phase Distribution
const dayBuckets: GeneratedSlotAssignment[][] = Array.from({ length: numDays }, () => []);
const daySubjectCounts: number[][] = Array.from({ length: numDays }, () => Array(subjectRequests.length).fill(0));

const sortedRequestsWithOriginalIdx = subjectRequests
  .map((req: SubjectRequirement, origIdx: number) => ({ ...req, origIdx }))
  .sort((a, b) => b.count - a.count);

sortedRequestsWithOriginalIdx.forEach((req) => {
  let remaining = req.count;
  const item: GeneratedSlotAssignment = {
    subject: req.subject,
    teacherName: req.teacherName,
    teacherId: req.teacherId
  };

  const basePerDay = Math.floor(req.count / numDays);
  const maxForThisSubject = Math.max(
    maxPeriodsPerDayPerSubject,
    Math.ceil(req.count / numDays)
  );

  // Pass 1: Assign basePerDay to ALL days first (Ensures weekly coverage!)
  // If count = 5 & numDays = 5: basePerDay = 1. Every day gets 1 period.
  // If count = 10 & numDays = 5: basePerDay = 2. Every day gets 2 periods.
  for (let round = 0; round < basePerDay; round++) {
    for (let d = 0; d < numDays; d++) {
      if (remaining > 0 && dayBuckets[d].length < numPeriods) {
        dayBuckets[d].push(item);
        daySubjectCounts[d][req.origIdx]++;
        remaining--;
      }
    }
  }

  // Pass 2: Distribute remaining periods (K % D) to days with the lowest total load
  if (remaining > 0) {
    const candidateDays = Array.from({ length: numDays }, (_, d) => d)
      .filter(d =>
        daySubjectCounts[d][req.origIdx] ≤ basePerDay &&
        daySubjectCounts[d][req.origIdx] < maxForThisSubject &&
        dayBuckets[d].length < numPeriods
      )
      .sort((a, b) => {
        const loadDiff = dayBuckets[a].length - dayBuckets[b].length;
        if (loadDiff !== 0) return loadDiff;
        const offsetA = (a + req.origIdx * 2 + classSectionOffset) % numDays;
        const offsetB = (b + req.origIdx * 2 + classSectionOffset) % numDays;
        return offsetA - offsetB;
      });

    for (const d of candidateDays) {
      if (remaining ≤ 0) break;
      dayBuckets[d].push(item);
      daySubjectCounts[d][req.origIdx]++;
      remaining--;
    }

    if (remaining > 0) {
      for (let d = 0; d < numDays && remaining > 0; d++) {
        if (dayBuckets[d].length < numPeriods && daySubjectCounts[d][req.origIdx] < maxForThisSubject) {
          dayBuckets[d].push(item);
          daySubjectCounts[d][req.origIdx]++;
        }
      }
    }
  }
});
```

```
        remaining--;  
    }  
}  
}  
});
```


5.2 Backtracking Solver & Post-Pass Safety Filter

```
// AutoTimetableGeneratorModal.tsx - Backtracking Placement Solver
function solveDailyPlacement(
  items: GeneratedSlotAssignment[],
  slots: (GeneratedSlotAssignment | null)[],
  numPeriods: number,
  preferredSlotOrder: number[]
): boolean {
  const freq = new Map<string, number>();
  items.forEach(it => freq.set(it.subject, (freq.get(it.subject) || 0) + 1));
  const sorted = [...items].sort((a, b) => {
    const fDiff = (freq.get(b.subject) || 0) - (freq.get(a.subject) || 0);
    if (fDiff !== 0) return fDiff;
    return a.subject.localeCompare(b.subject);
  });

  function backtrack(idx: number): boolean {
    if (idx ≥ sorted.length) return true;

    const currentItem = sorted[idx];
    const validCandidateSlots: { slot: number; score: number }[] = [];

    for (let orderIdx = 0; orderIdx < preferredSlotOrder.length; orderIdx++) {
      const slot = preferredSlotOrder[orderIdx];
      if (slots[slot] !== null) continue;

      // RULE: Check left neighbor - same subject must NEVER be consecutive
      if (slot > 0 && slots[slot - 1] !== null && slots[slot - 1]!.subject === currentItem.subject) {
        continue;
      }

      // RULE: Check right neighbor - same subject must NEVER be consecutive
      if (slot < numPeriods - 1 && slots[slot + 1] !== null && slots[slot + 1]!.subject === currentItem.subject) {
        continue;
      }

      // RULE: Check distance from existing instances of this subject today (must be ≥ 2)
      let minDistance = 999;
      let hasTooClose = false;
      for (let s = 0; s < numPeriods; s++) {
        if (slots[s] !== null && slots[s]!.subject === currentItem.subject) {
          const dist = Math.abs(s - slot);
          if (dist < 2) {
            hasTooClose = true;
            break;
          }
          if (dist < minDistance) {
            minDistance = dist;
          }
        }
      }
      if (hasTooClose) continue;

      // Scoring: reward maximum separation between instances of the same subject
      const score = minDistance !== 999 ? (minDistance * 20 - orderIdx) : (100 - orderIdx);
      validCandidateSlots.push({ slot, score });
    }

    // Explore candidates with highest separation score first
    validCandidateSlots.sort((a, b) => b.score - a.score);

    for (const cand of validCandidateSlots) {
      slots[cand.slot] = currentItem;
      if (backtrack(idx + 1)) return true;
      slots[cand.slot] = null; // Backtrack
    }

    return false;
  }
}
```

```

    return backtrack(0);
}

// Post-Pass Safeguard: Eliminates any accidental adjacent duplicates via swap
function eliminateConsecutiveDuplicates(slots: (GeneratedSlotAssignment | null)[]) {
    const numPeriods = slots.length;
    for (let i = 0; i < numPeriods - 1; i++) {
        const cur = slots[i];
        const next = slots[i + 1];
        if (cur && next && cur.subject === next.subject) {
            for (let j = 0; j < numPeriods; j++) {
                if (j === i || j === i + 1) continue;
                const other = slots[j];

                const otherValidAtIPlus1 = (!other || other.subject !== cur.subject) &&
                    (i + 2 ≥ numPeriods || !slots[i + 2] || !other || slots[i + 2]!.subject !== other.subject);

                const nextValidAtJ = (j === 0 || !slots[j - 1] || slots[j - 1]!.subject !== next.subject) &&
                    (j === numPeriods - 1 || !slots[j + 1] || slots[j + 1]!.subject !== next.subject);

                if (otherValidAtIPlus1 && nextValidAtJ) {
                    slots[i + 1] = other;
                    slots[j] = next;
                    break;
                }
            }
        }
    }
}

```

6. Step-by-Step Scheduling Scenarios & Algorithm Traces

Scenario 1: Subject with 5 Periods / Week (5 Working Days)

Parameters: Subject = "Mathematics", Weekly Quota K = 5, Days D = 5, Periods P = 8.
Phase 1 Calculation: $\text{basePerDay} = \text{floor}(5 / 5) = 1$.
Phase 1 Execution: Mathematics is allocated to Day 0 (Mon), Day 1 (Tue), Day 2 (Wed), Day 3 (Thu), Day 4 (Fri).
Result: Exactly 1 period per day. Weekly coverage is 100%. Consecutive duplicates = 0.

Scenario 2: Subject with 10 Periods / Week (5 Working Days)

Parameters: Subject = "English Language", Weekly Quota K = 10, Days D = 5, Periods P = 8.
Phase 1 Calculation: $\text{basePerDay} = \text{floor}(10 / 5) = 2$.
Daily Backtracking Solver Placement:

- Slot 0 (Period 1): English assigned.
- Slots 1 to 5: Other subjects assigned (Physics, Chemistry, History, etc.).
- Slot 6 (Period 7): English assigned (distance = 6 >= 2).

Result: Exactly 2 periods per day across all 5 days. Distance between sessions is 6 periods (Morning vs. Afternoon). Consecutive duplicates = 0.

Scenario 3: Real-World Mixed Curriculum (Social Studies 8, Chemistry 2)

DAY OF WEEK	PERIOD 1 (08:30)	PERIOD 2 (09:15)	PERIOD 3 (10:15)	PERIOD 4 (11:00)	PERIOD 5 (12:30)	PERIOD 6 (13:15)	PERIOD 7 (14:00)	PERIOD 8 (14:45)
Monday	SOCIAL STUDIES	CHEMISTRY	Mathematics	Physics	English	Biology	Physical Ed.	Library
Tuesday	SOCIAL STUDIES	Physics	Mathematics	English	Computer Sci.	SOCIAL STUDIES	Art	Music
Wednesday	Mathematics	SOCIAL STUDIES	English	CHEMISTRY	Physics	Biology	SOCIAL STUDIES	Moral Science
Thursday	SOCIAL STUDIES	English	Physics	Mathematics	Computer Sci.	SOCIAL STUDIES	Library	Physical Ed.
Friday	Physics	SOCIAL STUDIES	Mathematics	English	Biology	SOCIAL STUDIES	Art	Club Activity

7. Quality Assurance, Unit Testing & Build Verification

7.1 Automated Backend Unit Tests

Command: `dotnet test Backend.Tests --filter "FullyQualifiedName~TimetableServiceTests"`
Result: 15 Passed, 0 Failed, 0 Skipped (Total Duration: 2.14 seconds).

TEST CASE NAME	TARGET METHOD / CONSTRAINT	STATUS
GenerateTimetable_EnforcesWeeklyCoverageFirst	Two-Phase Base Allocation (floor(K / D))	PASSED
GenerateTimetable_PreventsBackToBackPeriods	Anti-Consecutive Rule (Subject_t != Subject_t-1)	PASSED
GenerateTimetable_PreventsTeacherClashes	Global Teacher Occupancy Tracker	PASSED
ValidateSlotConflicts_ThrowsOnTeacherOverlap	ValidateSlotConflictsAsync	PASSED
ValidateWeeklyLimit_ThrowsWhenQuotaExceeded	ValidateWeeklySubjectLimitAsync	PASSED
SaveTimetableSlot_ResolvesPeriodIdFromStartTime	Automatic Canonical Period ID Resolution	PASSED
SyncPeriodSettings_PreservesForeignKeyIntegrity	SyncPeriodSettingsAsync	PASSED

7.2 Production Build Verification

SUBSYSTEM	BUILD TOOLCHAIN	COMPILATION STATUS	ARTIFACT OUTPUT
Backend API	.NET SDK 8.0.404 / C# 12	0 ERRORS / 0 WARNINGS	Backend/bin/Debug/net8.0/Backend.dll
Frontend Web	Node v22 / TypeScript 5 / Vite 8	0 ERRORS / BUILT IN 10.74S	dist/assets/index-DhQEStSJ.js (5.1 MB)
Database Engine	MySQL 8.0 InnoDB (Port 3306)	OPERATIONAL	SMS_NEW Relational Database